GRUPO 3 · INVESTIGACIÓN GRUPAL

NVIDIA Tesla & RTX GPUs

Arquitectura, CUDA & Optimización

INTEGRANTES

George Roussel Briceño Celestino
Laura Elena Brenes Espinoza
Rodrigo Venegas Mora
Marlon Méndez Naranjo

INSTITUCIÓN

Instituto Tecnológico de Costa Rica

Temas: Ampere · Hopper · Blackwell · Modelo CUDA · CUDA-X · Optimización

¿Por qué GPUs? Computación Heterogénea

EL PROBLEMA

- **Power Wall** — el escalado de frecuencia se detuvo ~2005
- Las CPUs optimizan **latencia** con pocos núcleos potentes
- Las cargas modernas (IA, HPC) necesitan **throughput masivo**

LA SOLUCIÓN

- **Computación heterogénea:** CPU (host) + GPU (device)
- GPUs: miles de núcleos simples → máximo ancho de banda computacional
- Modelo **SIMT**: una instrucción, miles de hilos simultáneos

CPU VS GPU

CPU (HOST)

> Pocos núcleos complejos (4–64) · Cache enorme · Baja latencia · Control de flujo general

GPU (DEVICE)

> Miles de núcleos simples (1K–20K) · Alta BW de memoria · Alto throughput aritmético · SIMT

Clave: Offload de cómputo paralelo al GPU mientras CPU maneja lógica serial

Dos Líneas: Tesla / Data Center vs RTX

TESLA / DATA CENTER (A100, H100, B200)

- Sin salida de video — solo cómputo
- Memoria **HBM** (HBM2e / HBM3 / HBM3e) — máximo ancho de banda
- **NVLink** para interconexión multi-GPU de alta velocidad
- Soporte ECC completo · Multi-Instance GPU (MIG)
- Objetivo: HPC, entrenamiento e inferencia de IA

RTX (3090, 4090, 5090)

- Salida de video, HDMI/DP — workstations y gaming
- Memoria **GDDR6X** — alta BW a menor costo
- **RT Cores** para ray tracing en tiempo real
- **Tensor Cores** para DLSS / neural rendering
- Objetivo: gráficos profesionales + aceleración de IA local

Convergencia: Ambas líneas integran **Tensor Cores** · La frontera entre Data Center y workstation se difumina en generaciones recientes (Blackwell)

Arquitectura Ampere (2020)

A100 — DATA CENTER

- **54.2 B transistores** · nodo Samsung 7nm
- **Tensor Cores 3ª gen** — soporte FP16 / BF16 / TF32
- **Structured sparsity 2:4** — doble rendimiento efectivo
- **Multi-Instance GPU (MIG)** — hasta 7 instancias aisladas
- HBM2e a 2 TB/s · NVLink 3ª gen a 600 GB/s

GA102 — RTX 3090

- FP32 ALUs **duplicados** por SM vs Turing
- GDDR6X a **19.5 Gbps** · 936 GB/s
- 2ª gen RT Cores · DLSS 2.0

NOVEDADES CLAVE

TF32 — NUEVO FORMATO

Misma precisión que FP32 para DL, 10× más rápido. Transparente para frameworks.

STRUCTURED SPARSITY

Patrón 2:4 (2 de cada 4 valores distintos de cero). Tensor Core omite cálculos en ceros → ×2 throughput.

MIG — MULTI-INSTANCE GPU

Particionamiento físico del GPU con QoS garantizada. Aísla SMs, caché L2 y memoria HBM2e.

Arquitectura Hopper (2022)

H100 — DATA CENTER

- **80 B transistores** · TSMC 4N
- **Tensor Cores 4ª gen** con Transformer Engine
- **FP8 dinámico** — precisión mixta automática por capa
- HBM3 a **3.35 TB/s**
- **NVLink 4ª gen** a 900 GB/s · NVSwitch 3ª gen
- MIG 2ª gen — hasta 7 instancias con más granularidad

TRANSFORMER ENGINE

¿QUÉ HACE?

Detecta automáticamente la precisión óptima (FP8 / FP16 / BF16) por capa. Ideal para LLMs y modelos Transformer.

H200 (2024)

- Mismo die GH100 + **HBM3e** a 4.8 TB/s
- 141 GB de memoria vs 80 GB del H100
- Objetivo: inferencia de LLMs con contextos largos

NVL72: 72× H100 conectados con NVSwitch → sistema de 1 exaFLOP

Arquitectura Blackwell (2024-2025)

B200 — DATA CENTER

- **208 B transistores** · diseño multi-die
- **NV-HBI** die-to-die a **10 TB/s** — rompe el límite litográfico
- **Tensor Cores 5ª gen** — FP4 / NVFP4
- HBM3e · 8 TB/s ancho de banda
- **TMA** (Tensor Memory Accelerator) — mover datos shared↔global sin intervención de warps

RTX BLACKWELL (5090)

- **Neural Shading** — sombreado asistido por IA
- 4ª gen RT Cores · DLSS 4 con Multi-Frame Generation
- GDDR7 a ~28 Gbps

CAMBIO DE PARADIGMA

MULTI-DIE (CHIPLET)

Dos dies GB100 unidos por NV-HBI. Sistema operativo los ve como un solo GPU. Supera la barrera de tamaño de wafer.

FP4 / NVFP4

4 bits de precisión para inferencia → 2× throughput vs FP8. Requiere cuantización post-entreno.

Modelo de Programación CUDA

SISTEMA HETEROGÉNEO

- **Host (CPU)** — control, lógica serial, I/O
- **Device (GPU)** — kernels paralelos masivos
- Cada uno con su propia memoria

FLUJO DE EJECUCIÓN

1. malloc + inicialización en host
2. **cudaMemcpy** host → device
3. CPU lanza el **kernel** `kernel<<<grid,block>>>()`
4. GPU ejecuta en paralelo (SIMT)
5. **cudaDeviceSynchronize()**
6. cudaMemcpy device → host

JERARQUÍA DE HILOS

THREAD → BLOCK → GRID

Hasta 1024 threads/block · Bloques en 1D/2D/3D · Variables: threadIdx, blockIdx, blockDim, gridDim

WARPS (HARDWARE)

32 threads → 1 warp → 1 SM · Ejecución SIMT · Context switch oculta latencia de memoria

JERARQUÍA DE MEMORIA

Nivel	Alcance	Latencia
Registros	Por thread	~1 ciclo
Shared Mem	Por block	~5 ciclos
L1 / L2	SM / GPU	~30 ciclos
Global (HBM)	Todo el GPU	~600 ciclos

Ejemplo de Código: Adición de Vectores en CUDA C++

vecAdd.cu — CUDA C++ Programming Guide [3]

```cpp
#include <stdio.h>

// Kernel: cada thread suma un elemento
__global__ void vecAdd(float* A, float* B, float* C, int N) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) C[i] = A[i] + B[i];
}

int main() {
    int N = 1024;  size_t size = N * sizeof(float);
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);  cudaMalloc(&d_B, size);  cudaMalloc(&d_C, size);
    // ... inicializar + cudaMemcpy host→device ...
    int tpb = 256, bpg = (N + tpb - 1) / tpb;
    vecAdd<<<bpg, tpb>>>(d_A, d_B, d_C, N);
    cudaDeviceSynchronize();
    // ... cudaMemcpy device→host + cudaFree ...
}
```

PUNTOS CLAVE

- `__global__` — ejecuta en GPU, se llama desde CPU
- `blockDim.x*blockIdx.x+threadIdx.x` — índice global único
- `<<<bloques, hilos>>>` — configura la grilla de ejecución
- `cudaMalloc/Free` — gestión de memoria del device
- `cudaDeviceSynchronize` — barrera de sincronización

1024 elementos → 4 bloques × 256 hilos → cada hilo suma 1 par

Ecosistema de Software CUDA

CAPAS DEL STACK

Capa	Componentes
Frameworks	PyTorch · TensorFlow · JAX
CUDA-X Libs	cuBLAS · cuDNN · TensorRT · NCCL · Thrust
Runtime API	cudaMalloc · cudaMemcpy · kernel <<<>>>
Driver API	cuCtxCreate · cuModuleLoad
NVCC	CUDA C++ → PTX → SASS
Hardware	CUDA Cores · Tensor Cores · SMs

LENGUAJES SOPORTADOS

CUDA C/C++ **CUDA Python 1.0** **CUDA Fortran** OpenACC

Desde CUDA 13.3, Python es lenguaje oficialmente soportado para toda la plataforma [5]

HERRAMIENTAS DE DESARROLLO

- **NVCC** — compilador CUDA C++ (basado en LLVM)
- **Nsight Systems** — perfilador sistema CPU+GPU
- **Nsight Compute** — análisis detallado de kernels
- **CUDA-GDB** — depurador para código GPU

Principales Bibliotecas CUDA-X

Biblioteca	Dominio	Descripción
cuBLAS	Álgebra Lineal	Implementación GPU de BLAS (Basic Linear Algebra Subprograms) — gemm, axpy, etc.
cuDNN	Deep Learning	Primitivas optimizadas: convoluciones, normalización, activaciones, RNNs, atención
TensorRT / TRT-LLM	Inferencia	Optimizador e infraestructura runtime; soporta FP32/FP16/BF16/FP8/INT8; fusión automática
NCCL	Multi-GPU	Comunicación colectiva (AllReduce, Broadcast) de baja latencia entre GPUs y nodos
Thrust	Algoritmos	Biblioteca C++ estilo STL: sort, scan, reduce — paralelos en GPU, sin CUDA explícito
CUTLASS	GEMM / Tensor Cores	Plantillas C++ modulares y DSL Python para kernels matriciales de alto rendimiento
cuFFT	Transformadas	FFT acelerada en GPU — fundamental en procesamiento de señales e imágenes

Patrones de Optimización CUDA

COALESCENCIA DE MEMORIA GLOBAL

- Warp accede a **direcciones contiguas** → pocas transacciones
- Stride / accesos dispersos → una transacción por hilo → cuello de botella
- Solución: cargar a **shared memory**, reordenar, leer localmente [6]

GEMM POR TESELAS (TILED)

- Partir operandos en bloques que caben en shared mem / registros
- Datos se reutilizan → reduce tráfico a memoria global
- **cuBLAS / CUTLASS** implementan esto + adaptado a Tensor Cores [4][6]
- **Split-K:** reparte la reducción entre bloques → llena GPU con matrices delgadas

FUSIÓN DE KERNELS

CONV + RELU → 1 KERNEL

Elimina buffers intermedios en memoria global. TensorRT detecta y aplica automáticamente. Reduce latencia y ancho de banda. [8]

BATCHING Y CUDA GRAPHS

BATCHING

Procesar múltiples entradas a la vez → más trabajo útil por kernel. En TensorRT, lotes mayores casi siempre mejoran la eficiencia.

CUDA GRAPHS

Captura la secuencia completa de lanzamientos → reutiliza el grafo. Paga overhead de CPU una sola vez. Ideal para topologías fijas. [8]

Conclusiones y Tendencias

CONCLUSIONES

- **Hiperespecialización:** Tesla para HPC/IA, RTX para gráficos — ambas convergen en Tensor Cores
- **Salto generacional claro:** Ampere → Hopper → Blackwell con mejoras en BW, precisión y aislamiento
- **CUDA como ecosistema dominante:** de CUDA C++ a Python 1.0 — barrera de acceso se reduce
- **Optimización = balance:** cómputo + BW de memoria + overhead de lanzamiento

TENDENCIAS

- **Chiplets / multi-die:** Blackwell rompe límite litográfico con NV-HBI
- **FP4 / FP8:** precisión reducida para inferencia de LLMs a escala
- **TMA:** mover datos sin intervención de warps → libera recursos de cómputo
- **CUDA Python 1.0:** Python como lenguaje de primera clase en la plataforma

REFERENCIAS PRINCIPALES

[3] NVIDIA CUDA C++ Programming Guide — docs.nvidia.com/cuda/cuda-c-programming-guide
[4] NVIDIA CUDA-X Libraries — developer.nvidia.com/gpu-accelerated-libraries
[6][7] Ampere / Blackwell Tuning Guides — docs.nvidia.com/cuda/
[8] Optimizing TensorRT Performance — docs.nvidia.com/deeplearning/tensorrt